

Validité et terminaison

William Hergès
Sorbonne Université
william@herges.fr

Table des matières

1. Définitions élémentaires	2
2. Problèmes sur un algorithme	4
2.1. Exemple	4
2.2. Méthode générale	5
2.3. Autre exemple	6

1. Définitions élémentaires

Un algorithme sert à résoudre un problème.

△ Définition

Un *problème* P est identifié par un nom, un ensemble de paramètres et une description de la solution.

△ Définition

Une *instance* d'un problème P est une valeur possible pour les paramètres du problème.

Définition du problème de tri :

- nom : tri
- paramètres : un entier n et un tableau `tab` de 0 à $n - 1$ de tailles n
- solution : le tableau `tab` trié par ordre croissant

△ Définition

Un *programme* est composé de :

- un ensemble fini de variables
- des opérations élémentaires portant sur les variables (arithmétiques, logiques, affectations)
- des structures de contrôle permettant de définir un ordre sur les opérations élémentaires

△ Définition

Un *algorithme* qui résout un problème P est un programme qui vérifie les deux propriétés suivantes :

- **terminaison** : l'algorithme appliqué à toute instance du problème effectue un nombre fini d'opérations
- **validité** : l'algorithme résout le problème P pour toute instance de P

Un algorithme doit donc produire une solution en un temps fini qui fonctionne toujours.

On parle aussi de *correction* pour la validité.

△ Définition

Une *fonction* est un algorithme qui renvoie une valeur.

2. Problèmes sur un algorithme

Soit A un algorithme pour résoudre un problème P .

- Se termine-t-il ? → terminaison (cette séance)
- Résoud-il P ? → validité (cette séance)
- Est-il efficace ? → complexité (séance 3)

2.1. Exemple

Soit le programme

```
def fibo(n):
    if n == 0:
        return 0
    x = 0
    y = 1
    i = 1
    while i < n:
        z = x+y
        x = y
        y = z
        i = i+1
    return y
```

On cherche à savoir s'il calcule bien le n -ième terme de la suite de Fibonacci.

Le corps de boucle est composé d'instructions élémentaires et elle est effectuée $n - 1$ fois. Donc elle se termine pour tout n dans $\mathbb{N}$.

Pour vérifier la correction du programme, on va suivre les variables présentes dans le programme. Cela revient à faire du débogage. Un bon point d'arrêt est en fin de corps de boucle. Par exemple, on pourrait avoir :

i	x_i	y_i
1	0	1
2	1	1
3	1	2
4	2	3
5	3	5
6	5	8

On obtient alors 8 pour $n = 6$.

Pour la démontrer proprement, on la fait par récurrence. Soit $\mathcal{P}$ la propriété telle que :

$$\forall i \in \llbracket 1, n \rrbracket, \quad \mathcal{P}_i : \quad x_i = F_{i-1} \wedge y_i = F_i$$

On a que $\mathcal{P}_1$ est vérifiée.

Soit $k \in \mathbb{N}^*$ tel que $\mathcal{P}_k$ soit vraie.

$$y_{k+1} = x_k + y_k = F_k + F_{k-1} = F_{k+1}$$

et

$$x_{k+1} = y_k = F_k$$

Donc, $\mathcal{P}_{k+1}$ est vraie.

Ainsi, pour tout n dans $\mathbb{N}$, $\mathcal{P}$ est vraie.

manque la fin de la démo

2.2. Méthode générale

▲ Attention

Nous n'avons pas besoin d'un raisonnement par récurrence pour montrer la terminaison d'un programme.

△ Définition

Un *invariant de boucle* est une propriété qui est vérifiée à chaque exécution du corps de cette boucle. Elle est toujours vraie à la fin de la boucle, après avoir incrémentée notre i .

▲ Attention

Tous les invariants ne sont pas utiles !

Pour montrer la validité, on :

1. détermine un invariant de boucle
2. démontre l'invariant par récurrence sur le nombre d'itérations
3. étudie la sortie de boucle pour conclure sur la validité de la boucle

2.3. Autre exemple

On cherche l'indice de l'élément minimum d'un tableau. On propose ce programme :

```
def RechercheMin(tab):
    imin = 0
    i = 0
    while i < len(tab):
        if tab[i] < tab[imin]:
            imin[i]
        i = i+1
    return imin
```

Notons n la taille de `tab`.

Le nombre d'exécution de ce programme est borné. En effet, on exécute au maximum n fois le corps de la boucle.

Soit $\mathcal{P}$ la propriété suivante :

$$\forall i \in \llbracket 0, n \rrbracket, \quad \mathcal{P}_i : \quad \forall j \in \llbracket 0, i \rrbracket, \text{tab}[\text{imin}_i] \leq \text{tab}[j]$$

$\mathcal{P}_0$ est trivialement vraie.

Posons k dans $\llbracket 0, n \rrbracket$ telle que $\mathcal{P}_k$ soit vraie. On pose :

$$\text{imin}_{k+1} = \begin{cases} k+1 & \text{si } \text{tab}[k+1] < \text{tab}[\text{imin}_k] \\ \text{imin}_k & \text{sinon} \end{cases}$$

Or, comme $\mathcal{P}_k$ est vraie, on a $\text{tab}[i] \geq \text{tab}[\text{imin}_i]$ pour tout i dans $\llbracket 0, k \rrbracket$. De plus, on a $\text{tab}[k+1] \geq \text{tab}[\text{imin}_{k+1}]$ et $\text{tab}[\text{imin}_{k+1}] \leq \text{tab}[\text{imin}_k]$ par définition de imin_{k+1} . Donc $\mathcal{P}_{k+1}$ est vraie.

Ainsi, pour tout i dans $\llbracket 0, n \rrbracket$, $\mathcal{P}_i$ est vérifiée.

En sortie de boucle, $\mathcal{P}_n$ est vraie et renvoie imin_n . Comme $\mathcal{P}_n$ est vraie, $\text{tab}[\text{imin}_n]$ est le plus petit élément de `tab`. Alors, le programme est valide.